Projektsrapport

Projektuppgift
DT211G, Frontend-baserad webbutveckling

Pontus Johansson, pojo2001@student.miun.se
VT, 2026

Mittuniversitetet
MID SWEDEN UNIVERSITY

1 Sammanfattning

Denna rapport beskriver arbetsprocessen av webbapplikationen Solsving. Denna webbapplikation kombinerar väderdata och platsinformation för att hjälpa användaren att få en 7-dagars väderprognos samt en lista med de närmaste minigolfbanorna. Applikationen använder två stycken externa webbtjänster, Open-Mateo för väderdata och Geoapify Places API för platsdata. Projektet visar hela arbetsprocessen från planering och design till implementation. Resultatet blev en interaktiv och responsiv webbapplikation som presenterar väder och minigolfbanor, komplett med kartlänkar och fallback-logik för namn till banorna. Rapporten avslutas med en analys med tekniska val och alternativa förbättringar.

2 Innehållsförteckning

..... 1
1 Sammanfattning 2
2 Innehållsförteckning 3
3 Introduktion 4
4 Teori 5
5 Metod 6
6 Konstruktion 8
7 Resultat 11
8 Slutsats 12
9 Källförteckning 13

3 Introduktion

Syftet med projektarbetet var att utveckla en webbapplikation som som kombinerar flera API:er och skapa en användarvänlig tjänst. Användaren ska kunna söka på en plats och därefter få två typer av information.

- En lista över de närmaste minigolfbanorna
- En 7-dagars väderprognos.

Problemmotiveringen kommer från behovet att integrera flera datakällor, hantera asynkrona anrop och strukturera applikationen tydligt och tillgängligt. Projektet omfattar även felhantering, fallback-logik, modulär SASS-struktur och animationer.

4 Teori

För att förstår hur applikationen är uppbyggd kommer nedan en genomgång av tekniker och begrepp som använts under processen.

Fetch och asynkron JavaScript

Fetch används för att hämta data från ett API. Med async/await kan applikationen fortsätta köras medan den nya datan laddas, vilket förbättrar hela användarupplevelsen. [8]

Open-Meteo

En kostnadsfri vädertjänst som erbjuder en öppen API för väderprognoser baserat på koordinater. API:et kräver ingen API nyckel eller autentisering och är därför väldigt enkel att integrera. [5]

Geoapify Places API

En tjänst som returnerar platser utifrån kategori och geografiska filter. I detta projekt användes filter för att få fram uppgifter kring äventyrsgolf. [6]

SCSS

En CSS-preprocessor som möjliggör, partials, mixins, variabler och modulär struktur. Deta underlättar arbetsprocessen och organiserar projektet. [2]

Vite

Ett modernt byggverktyg som används för utvecklingsserver, optimering och produktion. Som automatiserar arbetsprocessen och komprimerar projektet. [1]

5 Metod

Projektet genomfördes genom en iterativ arbetsprocess där funktionalitet byggdes gradvid.

Följande metoder användes:

API-integration

Asynkrona anrop till Open-Meteo och Geoapify hämtar väderdata och platsinformation baserat på koordinater. [5, 6, 7]

DOM-manipulation

Allt innehåll på sidan genereras dynamiskt med JavaScript, inklusive loader, väderkort och golfkort.

Fallback-logik

Eftersom Geoapify ibland saknar namn på platser implementerades en fallback-kedja som följer. Om det finns ett namn på klubben så skrivs namnet ut, om inget namn finns så skrivs stadsdelen + bangolf ut, om ingen stadsdel är registrerad i places API:et skrivs ”Okänt namn” ut i kortet.

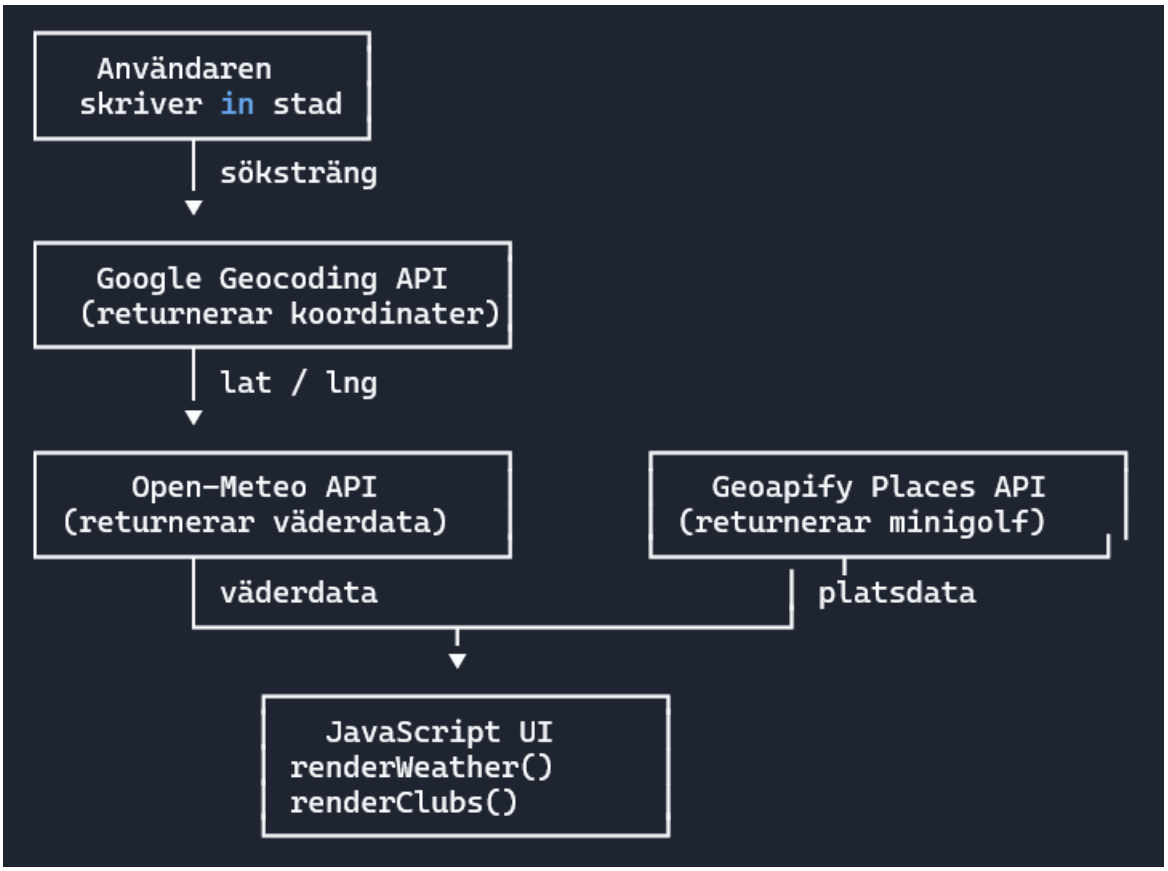
Modulär SCSS-struktur

Projektet organiserades i partials och döptes till [2]:

- bas-stil
- layout
- komponenter
- utilities
- UX-metoder
- Loader under API-anrop
- Fade-in-animationer
- Tydlig onboarding-text

Testning

Manuell testning genomförddes i flera olika webbläsare och enheter från mobila enheter och dator. Applikationens dataflöde visas i figur 1, där användarens söksträng först omvandlas till koordinater via Google Geocoding API. Dessa koordinater används sedan för att hämta väderdata oc platsdata med hjälp av Open-Mateo och Geoapify Places API.



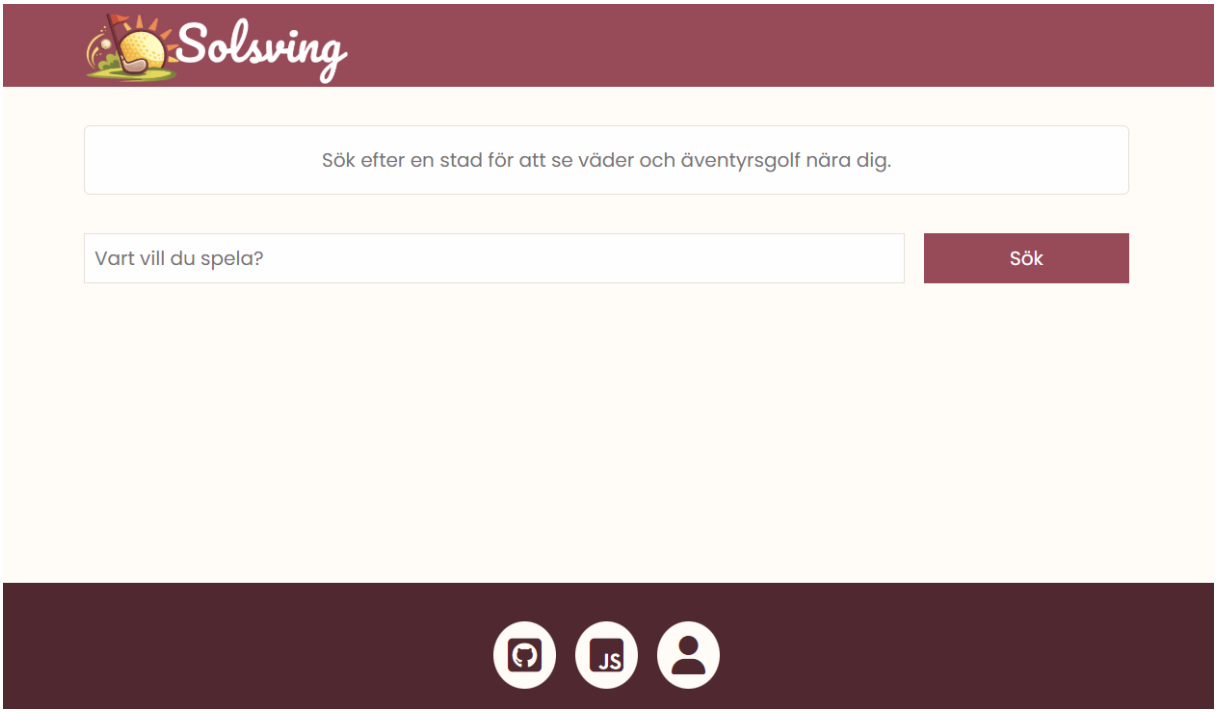
Figur 1. Översikt över applikationens dataflöde mellan användare, API:er och JavaScript-gränssnittet.

6 Konstruktion

Det praktiska arbetet delades upp i flera delar:

1. Sökflöde och användargränssnitt

Som visas i figur 2 möts användaren av en enkel startvy med ett sökfält och introduktionstext. Vid sökning döljs introduktionstexten och en loader visas.



Figur 2 - Startvy med sökfält och introduktionstext innan användaren gör en sökning.

2. API-anrop

Tre huvudfunktioner implementerades:

- searchLocation()
  - hämtar koordinater med hjälp av Google Geocoding
- searchWeather()
  - hämtar väderdata från Open-Meteo
- searchGolfClubs()
  - hämtar minigolfbanor från Geoapify Places API

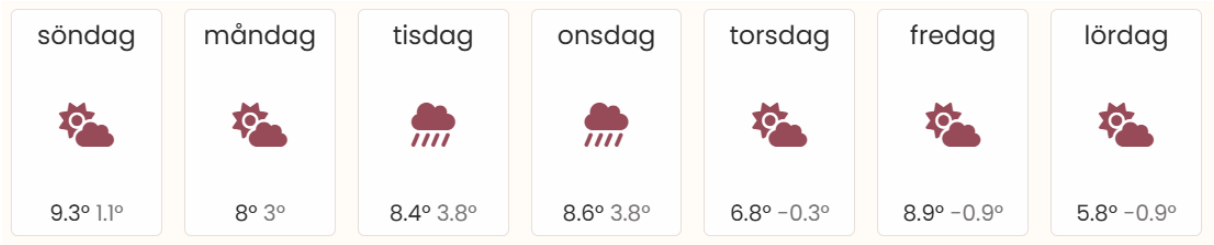
3. Rendering

Två renderingsfunktioner skapades:

- renderWeather()

- genererar 7 väderkort, detta visas i figur 4.

- renderClubs()
  - I figur 5 visas de genererade korten med namn, adress och kartlänk



Figur 4 – Generade väderkorten.

Figur 5 – Genererade kort för klubbar.

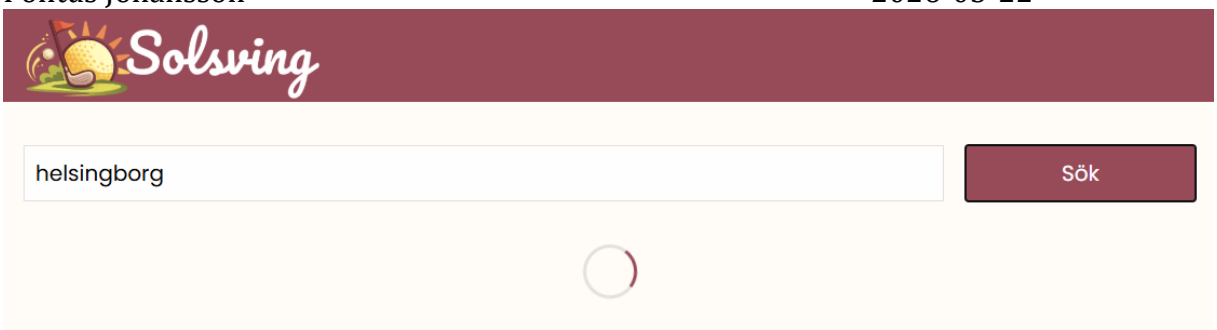
4. Fallback-logik

För att hantera saknade namn implementerades en prioriteringskedja som säkerställer att ett namn skrivs ut för golfbanorna. Om ett namn finns så skrivs namnet ut, om inget namn finns så skrivs stadsdelen + Bangolf ut, men om ingen stadsdel är registrerad så skrivs det ut ”Okänt namn”.

5. Animationer

Fade-in-effekter implementerades för väderkortet och golfkortet. [3]

- ta bort fade-in
- lägga till fade-in igen
- under API anrop visas en loader (figur 3) för att ge användaren visuell feedback.



Figur 3 – Loader som visas medans API-anrop genomförs.

6. SCSS-struktur

Projektet organiserades i:

- _base
- _animations
- _layout
- _components
- _utilities
- _reset
- Main

7.JSdocs

All JavaScript-kod dokumenterades med JSdocs [4], och publicerades med länk från webb-applikationen.

7 Resultat

Projektet i en fungerande webbapplikation som använder sig av tre stycken olika API:er som samarbete och uppfyller samtliga krav:

- Användaren kan söka efter en stad
- 7-dagars väderprognos visas
- Minigolfbanor listas med namn, adress och kartlänk
- Loader visas under API-anrop
- Resultat fade:ar in efter laddning
- Fallback-namn genereras korrekt
- Applikationen är responsiv och fungerar på mobil

Alla problem från problemmotiveringen har lösts.

8 Slutsats

Projektet visar styrkan i att kombinera olika API:er för att tillsammans skapa en webb-applikation som är skräddarsydd för en viss målgrupp. Arbetet gick övergripande bra, särskilt integrationen av API:et Open-Meteo som var enkel att arbeta med. Geoapify krävde mer logik på fält som saknade logik, vilket ledde till utvecklingen av fallback-logiken.

Möjliga förbättringar:

- mer robust felhantering felsökning och errors.
- Cache för API lagring
- mer avancerad UI-design, t.ex mest attraktiva sökningar eller historik
- möjlighet att visa fler kategorier än minigolf

Projektet gav en fördjupad förståelse för API-integration, asynkron JavaScript och modern frontend-utveckling.

9 Källförteckning

[1] Vite. Vite - Next Generation Frontend Tooling. Tillgänglig från: https://vite.dev/ Hämtad 2026-03-20.

[2] Sass. Sass Guide – Variables. Tillgänglig från: https://sass-lang.com/guide/#variables Hämtad 2026-03-20.

[3] W3Schools. CSS3 Animations. Tillgänglig från: https://www.w3schools.com/css/css3_animations.asp Hämtad 2026-03-18.

[4] JSDoc. JSDoc Documentation. Tillgänglig från: https://jsdoc.app/ Hämtad 2026-03-18.

[5] Open-Meteo. Weather Forecast API. Tillgänglig från: https://open-meteo.com/ Hämtad 2026-03-20.

[6] Geoapify. Forward Geocoding API Documentation. Tillgänglig från: https://apidocs.geoapify.com/docs/geocoding/forward-geocoding/ Hämtad 2026-03-20.

[7] Google Developers. Geocoding API Overview. Tillgänglig från: https://developers.google.com/maps/documentation/geocoding/overview Hämtad 2026-03-20.

[8] JavaScript.info. Async/await. Tillgänglig från: https://javascript.info/async-await Hämtad 2026-03-18.